

Willkommen in der Mikrocontroller.net Artikelsammlung. Alle Artikel hier können nach dem Wiki-Prinzip von jedem bearbeitet werden. [Zur Hauptseite der Artikelsammlung](#)

STM32 BLDC Control with HALL Sensor

Aus der Mikrocontroller.net Artikelsammlung, mit Beiträgen verschiedener Autoren (siehe Versionsgeschichte)

The code is not a complete project, it's to show you how to use the motor timer to control an BLDC Motor in combination with an HALL Sensor on another timer.

Inhaltsverzeichnis

- [1 Intro](#)
- [2 Code for the HallSensor timer](#)
- [3 Code for BLDC motor control timer](#)
- [4 Some things good to know](#)
- [5 Footnotes](#)

Intro

The system needs two timers. One to control [PWM](#) for the bridge [FETs](#) and another timer to identify the HallSensor signals. The HallSensor timer triggers the motor timer commutation event.

The connection between these two timers is done in the background with events. There is no direct need for interrupt handling to commutate the motor timer.

- BLDC Bridge FETs: TIM1
- HallSensor: TIM4

Recommendation: For your first projects and to learn more, I suggest to use a protection 3 phase motor driver. In there documents you find good information about shoot-through protection, on-time, off-time, dead-time, hall-steps, rpm and field frequency, cutoff filter frequency and many more. Some of the chips offers also simplified usage of sensorless motor control because they detect the rotorposition and give back a virtual Hall Signal, see the TMC product.

- Allegro A4935 [1]
- TMC 603 [2]

If you have a simple FET bridge driver then you must carefully calculate the dead time which is a parameter in the motortimer.

Active Freewheeling

Without active freewheeling the body diodes during PWM OFF time produce a huge amount of power loss. With active freewheeling this power loss can be reduced.

Info

Check HallSensor Inputs. They often needs PullUps, use RC filters to filter bad signals.

Code for the HallSensor timer

Internal Connection from Hall/Enc Timer to Motor Timer. The HALL Timer Output is direct connected to the Motor Timer Commutation Trigger. If the correct combination of Motor and Hall/Enc Timer is selected then this is done for you. If you can not use the internal Connection you have to do this in an Interrupt manually.

Only following Combinations are possible^[1]

If Motor is on Timer1 this is possible

- Hall/Enc is Timer 2 → Motor is Timer 1 → use TIM_TS_ITR1
- Hall/Enc is Timer 3 → Motor is Timer 1 → use TIM_TS_ITR2
- Hall/Enc is Timer 4 → Motor is Timer 1 → use TIM_TS_ITR3
- Hall/Enc is Timer 5 → Motor is Timer 1 → use TIM_TS_ITR0

If Motor is on Timer8 this is possible

- Hall/Enc is Timer 1 → Motor is Timer 1 → use TIM_TS_ITR0
- Hall/Enc is Timer 2 → Motor is Timer 1 → use TIM_TS_ITR1
- Hall/Enc is Timer 4 → Motor is Timer 1 → use TIM_TS_ITR2
- Hall/Enc is Timer 5 → Motor is Timer 1 → use TIM_TS_ITR3

```
void configHallSensorTimer(void) {
```

```
// Timer 3 decodes the 3 HallSensor input lines  
// see referenze manual page 305
```

```
// define timer clock  
// between two changes on the hall sensor lines on the lowest rotation  
// speed (eg. 1/100 from max. speed) the timer must not overflow  
// define timer counter clock appropriate
```

```
// enable port pins for hall inputs  
RCC_APB2PeriphClockCmd(...);  
GPIO_InitStructure.GPIO_Pin = GPIO_Pin...;  
GPIO_InitStructure.GPIO_Mode = ...  
GPIO_Init(..., &GPIO_InitStructure);
```

```
RCC_APB1PeriphClockCmd(TIM4_CLK, ENABLE);
```

```
// timer base configuration
// 126 => 3,5s till overflow ; 285,714kHz TimerClock [36MHz/Prescaler]
TIM_TimeBaseStructure.TIM_Prescaler = 126;
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
TIM_TimeBaseStructure.TIM_Period = 65535;
TIM_TimeBaseStructure.TIM_ClockDivision = 0;
TIM_TimeBaseStructure.TIM_RepetitionCounter = 0;
TIM_TimeBaseInit(TIM4, &TIM_TimeBaseStructure);

// enable hall sensor
// TI1F_ED will be connected to HallSensoren Inputs
// TIM4_CH1, TIM4_CH2, TIM4_CH3
TIM_SelectHallSensor(TIM4, ENABLE);

// TIM_EncoderInterfaceConfig(TIM_TypeDef* TIMx, uint16_t TIM_EncoderMode,
// uint16_t TIM_IC1Polarity, uint16_t TIM_IC2Polarity)

// HallSensor event is delivered with signal TI1F_ED
// (this is XOR of the three hall sensor lines)
// Signal TI1F_ED: falling and rising edge of the inputs is used
TIM_SelectInputTrigger(TIM4, TIM_TS_TI1F_ED);

// On every TI1F_ED event the counter is resetted and update is triggered
TIM_SelectSlaveMode(TIM4, TIM_SlaveMode_Reset);

// Channel 1 in input capture mode
// on every TCR edge (build from TI1F_ED which is a HallSensor edge)
// the timervalue is copied into ccr register and a CCR1 Interrupt
// TIM_IT_CC1 is fired

TIM_ICInitStructure.TIM_Channel = TIM_Channel_1;
TIM_ICInitStructure.TIM_ICPolarity = TIM_ICPolarity_Rising;
// listen to T1, the HallSensorEvent
TIM_ICInitStructure.TIM_ICSelection = TIM_ICSelection_TRC;
// Div:1, every edge
TIM_ICInitStructure.TIM_ICPrescaler = TIM_ICPSC_DIV1;
// noise filter: 1111 => 72000kHz / factor (==1) / 32 / 8 -> 281kHz
// input noise filter (reference manual page 322)
TIM_ICInitStructure.TIM_ICFilter = 0xF;
TIM_ICInit(TIM4, &TIM_ICInitStructure);

// channel 2 can be use for commutation delay between hallsensor edge
// and switching the FET into the next step. if this delay time is
// over the channel 2 generates the commutation signal to the motor timer
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM2;
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
TIM_OCInitStructure.TIM_Pulse = 1; // 1 is no delay; 2000 = 7ms
TIM_OC2Init(TIM4, &TIM_OCInitStructure);
```

```

// clear interrupt flag
TIM_ClearFlag(TIM4, TIM_FLAG_CC2);

//TIM_SelectMasterSlaveMode(TIM4, TIM_MasterSlaveMode_Enable);
// TIM_SelectOutputTrigger(TIM4, TIM_TRGOSource_OC1);
// timer2 output compare signal is connected to TRIGO
TIM_SelectOutputTrigger(TIM4, TIM_TRGOSource_OC2Ref);

// Enable channel 2 compare interrupt request
// TIM_IT_CC1 | TIM_IT_CC2
TIM_ITConfig(TIM4, TIM_IT_CC1 | TIM_IT_CC2, ENABLE);

// Enable output compare preload
//TIM_OC4PreloadConfig(TIM4, TIM_OCPreLoad_Enable);

// Enable ARR preload
//TIM_ARRPreloadConfig(TIM4, ENABLE);

// Enable update event
//TIM_ClearFlag(TIM4, TIM_FLAG_Update);
//TIM_ITConfig(TIM4, TIM_IT_Update, DISABLE);

// we use preemption interrupts here, BLDC Bridge switching and
// Hall has highest priority
NVIC_InitStructure.NVIC_IRQChannel = TIM4_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0x00;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&NVIC_InitStructure);

// -----
// HallSensor is now configured, if BLDC Timer is also configured
// after enabling timer 4
// the motor will start after next overflow of the hall timer because
// this generates the first startup motor commutation event
TIM_Cmd(TIM4, ENABLE);
}

void incCommutationDelay(void) {
    TIM4->CCR2 = (TIM4->CCR2) + 1;
}

void decCommutationDelay(void) {
    TIM4->CCR2 = (TIM4->CCR2) - 1;
}

// ----- HallSensor interrupt handler -----

// this handles TIM4 irqs (from HallSensor)

```

```

void TIM4_IRQHandler(void) {
    if (TIM_GetITStatus(TIM4, TIM_IT_CC1) != RESET)
    {
        TIM_ClearITPendingBit(MOTOR_TMC603_HALLENC_TIM, TIM_IT_CC1);
        // calculate motor speed or else with CCR1 values
        hallccr1 = TIM4->CCR1;
        ...
    }
    else if (TIM_GetITStatus(TIM4, TIM_IT_CC2) != RESET)
    {
        TIM_ClearITPendingBit(MOTOR_TMC603_HALLENC_TIM, TIM_IT_CC2);
        // this interrupt handler is called AFTER the motor commutation event
        // is done
        // after commutation the next motor step must be prepared
        // use inline functions in irq handlers static __INLINE funct(..) {...}
        BLDCMotorPrepareCommutation();
    } else {
        ; // this should not happen
    }
}

```

Code for BLDC motor control timer

```

void configMotorBridgeTimer(void) {

    // define timer clock, motor timer can be TIM1 or TIM8
    RCC_APB2PeriphClockCmd(...);

    // define the 6 output pins for the bridge, if needed define
    // the input pin for emergency stop

    // Chopper Frequency (PWM for the FETs)
    // 18kHz was good in empiric tests
    // ARR = SystemCoreClock / ChopperFreq
    // ARR defines also the resolution of the Chopper PWM
    #define BLDC_CHOPPER_PERIOD ((uint16_t)4000)

    // Time Base configuration

    TIM_TimeBaseStructure.TIM_Prescaler = 0;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseStructure.TIM_Period = BLDC_CHOPPER_PERIOD;
    TIM_TimeBaseStructure.TIM_ClockDivision = 0;
    TIM_TimeBaseStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM1, &TIM_TimeBaseStructure);

    // Channel 1, 2, 3 – set to PWM mode - all 6 outputs
    // per channel on output is low side fet, the opposite is for high side fet

```

```
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_Timing;
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;
TIM_OCInitStructure.TIM_Pulse = 0; // BLDC_ccr_val

TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
TIM_OCInitStructure.TIM_OCNPolarity = TIM_OCNPolarity_High;
TIM_OCInitStructure.TIM_OCIdleState = TIM_OCIdleState_Set;
TIM_OCInitStructure.TIM_OCNIdleState = TIM_OCNIdleState_Set;

TIM_OC1Init(TIM1, &TIM_OCInitStructure);
TIM_OC2Init(TIM1, &TIM_OCInitStructure);
TIM_OC3Init(TIM1, &TIM_OCInitStructure);

// activate preloading the CCR register
TIM_OC1PreloadConfig(TIM1, TIM_OCPreload_Enable);
TIM_OC2PreloadConfig(TIM1, TIM_OCPreload_Enable);
TIM_OC3PreloadConfig(TIM1, TIM_OCPreload_Enable);

/* automatic output enable, break off, dead time ca. 200ns and

// no lock of configuration */

TIM_BDTRInitStructure.TIM_OSSRState = TIM_OSSRState_Enable;
TIM_BDTRInitStructure.TIM_OSSIState = TIM_OSSIState_Enable;
TIM_BDTRInitStructure.TIM_LOCKLevel = TIM_LOCKLevel_OFF;

// DeadTime value n=1 bis 31: from 14ns to 1,7us
// DeadTime value n=129 bis 159: from 1,7µs to 3,5ms
// DeadTime value 7 => 98ns
// ... see programmers reference manual

// DeadTime[ns] = value * (1/SystemCoreFreq) (on 72MHz: 7 is 98ns)
TIM_BDTRInitStructure.TIM_DeadTime = 7; // 98ns

TIM_BDTRInitStructure.TIM_AutomaticOutput = TIM_AutomaticOutput_Enable;

// enabel this if you use emergency stop signal
// TIM_BDTRInitStructure.TIM_Break = TIM_Break_Enable;
// TIM_BDTRInitStructure.TIM_BreakPolarity = MOTOR_TMC603_EMSTOP_POLARITY;

TIM_BDTRInitStructure.TIM_Break = TIM_Break_Disable;

TIM_BDTRConfig(TIM1, &TIM_BDTRInitStructure);

// preload ARR register
TIM_CCPreloadControl(TIM1, ENABLE);

// activate COM (Commutation) Event from Slave (HallSensor timer)
// through TRGI
```

```

enableHallCommutateSignal();

// Internal connection from Hall/Enc Timer to Motor Timer
// eg. TIM1 (BLDC Motor Timer) is Slave of TIM3 (Hall Timer)
// Internal connection from Hall/Enc Timer to Motor Timer

// Choose carefully from the following possible combination
// check programmers reference manual
// TIM_SelectInputTrigger(TIM1, TIM_TS_ITR0);
// MotorTimer = TIM1, HallTimer = TIM5
// TIM_SelectInputTrigger(TIM1, TIM_TS_ITR1);
// MotorTimer = TIM1, HallTimer = TIM2
// TIM_SelectInputTrigger(TIM1, TIM_TS_ITR2);
// MotorTimer = TIM1, HallTimer = TIM3
TIM_SelectInputTrigger(TIM1, TIM_TS_ITR3);
// MotorTimer = TIM1, HallTimer = TIM4
// TIM_SelectInputTrigger(TIM8, TIM_TS_ITR0);
// MotorTimer = TIM8, HallTimer = TIM1
// TIM_SelectInputTrigger(TIM8, TIM_TS_ITR1);
// MotorTimer = TIM8, HallTimer = TIM2
// TIM_SelectInputTrigger(TIM8, TIM_TS_ITR2);
// MotorTimer = TIM8, HallTimer = TIM4
// TIM_SelectInputTrigger(TIM8, TIM_TS_ITR3);
// MotorTimer = TIM8, HallTimer = TIM5

// Enable interrupt, motor commutation has high priority and has
// a higher subpriority then the hall sensor
NVIC_InitStructure.NVIC_IRQChannel = TIM1_TRG_COM_IRQn;

// highest priority
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0x00;

NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
// highest priority
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;

NVIC_Init(&NVIC_InitStructure);

// Interrupt for hardwired EmergencyStop
(if needed)
// Timer 1 Motor Emergency Break Input
// NVIC_InitStructure.NVIC_IRQChannel = TIM1_BRK_IRQn;
// NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0x00;
// NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
// NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
// NVIC_Init(&NVIC_InitStructure);

// ----- activate the bldc bridge ctrl. -----
// in a project this will be done late after complete
// configuration of other peripherie

```

```

// enable COM (commutation) IRQ
TIM_ITConfig(TIM1, TIM_IT_COM, ENABLE);

// enable motor timer
TIM_Cmd(TIM1, ENABLE);

// enable motor timer main output (the bridge signals)
TIM_CtrlPWMOutputs(TIM1, ENABLE);
}

// enable the connection between HallTimer and MotorTimer
void enableHallCommutateSignal() {
    TIM_SelectCOM(TIM1, ENABLE);
}

// disable the connection between HallTimer and MotorTimer
void disableHallCommutateSignal() {
    TIM_SelectCOM(TIM1, DISABLE);
}

// BLDC motor steps
// every row from 1 to 6 is called by a hall state
// every column a FET from 3-phase bridge
// motor off is at row 0 BLDC_BRIDGE_STATE_VORWARD[0]
// cw - rechtslauf - positiv
// { 1H,1L , 2H,2L , 3H,3L }
// BLDC motor steps
// every row from 1 to 6 is one of the 6 motor vector state
// every column a FET from 3-phase bridge
// all FETs off at row 0 or 8 (this pattern should not come from the hallsensor)
// cw - rechtslauf - positiv
// { 1H,1L , 2H,2L , 3H,3L }
static const uint8_t BLDC_BRIDGE_STATE_VORWARD[8][6] = // Motor step
{
    { FALSE,FALSE , FALSE,FALSE , FALSE,FALSE }, // 0
    { FALSE,TRUE , TRUE ,FALSE , FALSE,FALSE }, // 2
    { FALSE,FALSE , FALSE,TRUE , TRUE ,FALSE }, // 4
    { FALSE,TRUE , FALSE,FALSE , TRUE ,FALSE }, // 3
    { TRUE ,FALSE , FALSE,FALSE , FALSE,TRUE } // 6
    { FALSE,FALSE , TRUE ,FALSE , FALSE,TRUE }, // 1
    { TRUE ,FALSE , FALSE,TRUE , FALSE,FALSE }, // 5
    { FALSE,FALSE , FALSE,FALSE , FALSE,FALSE }, // 0
};

// This function handles motor timer trigger and commutation interrupts
// can be used for calculation...
void TIM1_TRG_COM_IRQHandler(void)
{
    TIM_ClearITPendingBit(TIM1, TIM_IT_COM);
}

```



```

    // commutationCount++;
}

/* This is called from HALL timer interrupt handler
remember:
    if hall a hall edge is detected
    first the motor commutation event is done
    next this routine is called which has to prepare the next motor step
    (which FET must be switched on or off)
    active freewheeling is used to minimize power loss

code should be easy to understand and to debug... for practical use
you should optimize it */

```

```

static __INLINE void BLDCMotorPrepareCommutation(void)
{
    // next bridge step calculated by HallSensor inputs
    // if there was an hall event without changing the hall position,
    // do nothing.
    //
    // In principle, on every hall event you can go to the next motor
    // step but i had sometimes problems that the motor was running
    // on an harmonic wave (??) when the motor was without load
    uint16_t newhallpos = ((GPIO_ReadInputData(GPIOD) & 0x7000) >> 12);

    if (newhallpos == hallpos) return;
    lasthallpos = hallpos;

    hallpos = newhallpos;

    // this is only for motor direction forward

    BH1 = BLDC_BRIDGE_STATE_VORWARD[hallpos][0];
    BL1 = BLDC_BRIDGE_STATE_VORWARD[hallpos][1];

    BH2 = BLDC_BRIDGE_STATE_VORWARD[hallpos][2];
    BL2 = BLDC_BRIDGE_STATE_VORWARD[hallpos][3];

    BH3 = BLDC_BRIDGE_STATE_VORWARD[hallpos][4];
    BL3 = BLDC_BRIDGE_STATE_VORWARD[hallpos][5];

    // **** this is with active freewheeling ****

    // Bridge FETs for Motor Phase U
    if (BH1) {

        // PWM at Low side FET of bridge U
        // active freewheeling at high side FET of bridge U
        // if low side FET is in PWM off mode then the high side FET
        // is ON for active freewheeling. This mode needs correct definition
    }
}

```

```
// of dead time otherwise we have shoot-through problems

TIM_SelectOCxM(TIM1, TIM_Channel_1, TIM_OCMode_PWM1);

TIM_CCxCmd(TIM1, TIM_Channel_1, TIM_CCx_Enable);

TIM_CCxNCmd(TIM1, TIM_Channel_1, TIM_CCxN_Enable);

} else {

// Low side FET: OFF
TIM_CCxCmd(TIM1, TIM_Channel_1, TIM_CCx_Disable);

if (BL1){

// High side FET: ON
TIM_SelectOCxM(TIM1, TIM_Channel_1, TIM_ForcedAction_Active);

TIM_CCxNCmd(TIM1, TIM_Channel_1, TIM_CCxN_Enable);

} else {

// High side FET: OFF
TIM_CCxNCmd(TIM1, TIM_Channel_1, TIM_CCxN_Disable);

}

}

// Bridge FETs for Motor Phase V

if (BH2) {
TIM_SelectOCxM(TIM1, TIM_Channel_2, TIM_OCMode_PWM1);
TIM_CCxCmd(TIM1, TIM_Channel_2, TIM_CCx_Enable);
TIM_CCxNCmd(TIM1, TIM_Channel_2, TIM_CCxN_Enable);
} else {
TIM_CCxCmd(TIM1, TIM_Channel_2, TIM_CCx_Disable);

if (BL2){
TIM_SelectOCxM(TIM1, TIM_Channel_2, TIM_ForcedAction_Active);
TIM_CCxNCmd(TIM1, TIM_Channel_2, TIM_CCxN_Enable);
} else {
TIM_CCxNCmd(TIM1, TIM_Channel_2, TIM_CCxN_Disable);
}
}

// Bridge FETs for Motor Phase W

if (BH3) {
TIM_SelectOCxM(TIM1, TIM_Channel_3, TIM_OCMode_PWM1);
```

```
TIM_CCxCmd(TIM1, TIM_Channel_3, TIM_CCx_Enable);
TIM_CCxNCmd(TIM1, TIM_Channel_3, TIM_CCxN_Enable);
} else {
    TIM_CCxCmd(TIM1, TIM_Channel_3, TIM_CCx_Disable);

    if (BL3){
        TIM_SelectOCxM(TIM1, TIM_Channel_3, TIM_ForcedAction_Active);
        TIM_CCxNCmd(TIM1, TIM_Channel_3, TIM_CCxN_Enable);
    } else {
        TIM_CCxNCmd(TIM1, TIM_Channel_3, TIM_CCxN_Disable);
    }
}
}
```

Some things good to know

// You can manually generate an commutation event (like the hall sensor)

```
TIM_GenerateEvent(TIM1, TIM_EventSource_COM);
```

// how to change the PWM value

```
TIM1->CCR1 = new_ccr_val;
```

```
TIM1->CCR2 = new_ccr_val;
```

```
TIM1->CCR3 = new_ccr_val;
```

Footnotes

1. Check STM32 Reference Manual for Internal Trigger 1 to 4 (ITR1 to ITR4). See Table "TIMx Internal trigger connection".

Kategorie:

- [STM32](#)